

PART 1: DUCKDB ESSENTIALS

FROM-First Queries

```
-- No SELECT needed
FROM my_table;
FROM my_table LIMIT 10;

-- Read files directly
FROM 'data.csv';
FROM 'data.parquet';
FROM 'https://example.com/data.parquet';

-- Glob patterns
FROM 'data/*.parquet';
FROM read_parquet(
  's3://bucket/path/**/*.parquet');
```

SELECT Modifiers

```
-- Exclude columns
SELECT * EXCLUDE (sensitive_col, internal_id)
FROM users;

-- Replace columns inline
SELECT * REPLACE (lower(name) AS name,
  age + 1 AS age) FROM users;

-- Rename columns in wildcard
SELECT * RENAME (name AS user_name)
FROM users;

-- Select columns by regex pattern
SELECT COLUMNS('sales_*') FROM sales_data;

-- Apply function to matched columns
SELECT MIN(COLUMNS('sales_*')) FROM sales_data;

-- Select by position
SELECT #1, #3 FROM my_table;

-- DISTINCT ON (first row per group)
SELECT DISTINCT ON (country)
  country, city, population
FROM cities ORDER BY population DESC;
```

GROUP BY ALL / ORDER BY ALL

```
SELECT country, city, SUM(sales)
FROM orders GROUP BY ALL;
SELECT * FROM my_table ORDER BY ALL;
```

Function Chaining (Dot Syntax)

```
SELECT name.upper().replace(' ', '_') FROM users;
SELECT ('hello world').upper();
SELECT col.trim().lower().length() FROM t;
```

Friendly SQL

```
CREATE OR REPLACE TABLE t
  AS SELECT 1 AS id, 'a' AS val;
CREATE TABLE IF NOT EXISTS t (id INT, val TEXT);

-- Upsert
INSERT INTO t VALUES (1, 'updated')
ON CONFLICT (id)
DO UPDATE SET val = EXCLUDED.val;
INSERT OR REPLACE INTO t VALUES (1, 'updated');
INSERT OR IGNORE INTO t VALUES (1, 'skipped');

-- Column aliases usable in WHERE/GROUP BY/HAVING
SELECT a + b AS total FROM t WHERE total > 10;
```

File I/O

```
-- Reading
SELECT * FROM read_csv('data.csv',
  header = true, auto_detect = true);
SELECT * FROM read_parquet('data.parquet');
SELECT * FROM read_json('data.json');

-- Writing
COPY my_table TO 'output.parquet'
(FORMAT PARQUET);
COPY my_table TO 'output.csv'
(HEADER, DELIMITER ',');
COPY (SELECT * FROM t WHERE x > 10)
TO 'filtered.parquet' (FORMAT PARQUET);

-- Partitioned export
COPY orders TO 'out'
(FORMAT PARQUET, PARTITION_BY (year, month));

-- Direct insert from file / Create table from file
INSERT INTO my_table
SELECT * FROM read_csv('new_data.csv');
CREATE TABLE t AS FROM 'data.parquet';
```

Data Types & Complex Types

```
-- LIST (variable-length array)
SELECT [1, 2, 3] AS my_list;
-- l-indexed access and slicing
SELECT my_list[1];
SELECT my_list[1:2];
-- Expand list to rows
SELECT unnest([1, 2, 3]);

-- STRUCT (named fields)
SELECT ('name': 'Alice', 'age': 30) AS person;
-- Dot or bracket access
SELECT person.name;
SELECT person['age'];

-- MAP (key-value pairs)
SELECT MAP {'a': 1, 'b': 2} AS m;
SELECT m['a'];
SELECT map_keys(m), map_values(m);

-- UNION (tagged union type)
CREATE TABLE t (value UNION(str TEXT, num INT));
-- Returns 'str' or 'num'
SELECT union_tag(value) FROM t;

-- VARIANT (semi-structured, v1.4+)
SELECT 42::VARIANT;
SELECT variant_typeof(v);
SELECT variant_extract(v, 'INTEGER');
```

```
-- UNNEST: list to rows, struct to columns
SELECT unnest([1, 2, 3]) AS val;
SELECT unnest(['a': 1, 'b': 'hello']);
SELECT unnest([( 'a': 1), ( 'a': 2)],
  recursive := true);
-- WITH ORDINALITY (v1.4+)
SELECT * FROM unnest(['a', 'b', 'c'])
  WITH ORDINALITY;
```

```
-- Casting
SELECT '42'::INTEGER;
-- Returns NULL on failure
SELECT TRY_CAST('abc' AS INTEGER);
```

List Operations & Lambdas

```
-- Lambda syntax (preferred since v1.5)
-- [2, 4, 6]
SELECT list_transform([1, 2, 3],
  lambda x: x * 2);
-- [3, 4]
SELECT list_filter([1, 2, 3, 4],
  lambda x: x > 2);
-- 10
SELECT list_reduce([1, 2, 3, 4],
  lambda x, y: x + y);

-- Key list functions
SELECT list_sort([3, 1, 2]);
SELECT list_distinct([1, 1, 2, 3]);
SELECT list_concat([1, 2], [3, 4]);
SELECT list_contains([1, 2, 3], 2);
SELECT flatten([[1, 2], [3, 4]]);
SELECT list_aggregate([1, 2, 3], 'sum');
SELECT list_cosine_similarity([1,0], [0,1]);

-- Generate sequences
SELECT range(5);
SELECT unnest(generate_series(1, 10));
```

Struct Operations

```
SELECT struct_insert({'a': 1, b := 2});
SELECT struct_concat({'a': 1}, {'b': 2});
SELECT struct_keys({'a': 1, 'b': 2});
SELECT struct_values({'a': 1, 'b': 2});
-- v1.4+
SELECT struct_contains({'a': 1}, 'a');
SELECT struct_update({'a': 1, 'b': 2},
  b := 99);
```

String Functions

```
SELECT 'Hello' || ' ' || 'World';
SELECT length('DuckDB');
SELECT upper('duck'), lower('DUCK');
SELECT contains('DuckDB', 'Duck');
SELECT starts_with('DuckDB', 'Duck');
SELECT string_split('a,b,c', ',');
-- Slicing (l-indexed)
SELECT 'DuckDB'[1:4];

-- Regex
SELECT regexp_extract('abc123', '(\d+)', 1);
SELECT regexp_replace('hello world',
  'world', 'DuckDB');
SELECT regexp_matches('test123', '\d+');
SELECT regexp_extract_all('alb2c3', '\d+');
```

```
-- Formatting
SELECT format('{} has {} rows', 'tbl', 1000);
SELECT lpad('42', 5, '0');
SELECT trim(' hello ');
```

Date/Time Functions

```
SELECT current_date, current_timestamp, now();
SELECT DATE '2024-01-15';
SELECT make_date(2024, 1, 15);
SELECT make_timestamp(2024, 1, 15, 10, 30, 0);

-- Extraction
SELECT date_part('year', DATE '2024-06-15');
SELECT extract(month FROM TIMESTAMPT '2024-06-15');
SELECT dayname(DATE '2024-06-15');

-- Arithmetic
SELECT DATE '2024-01-01' + INTERVAL 30 DAY;
SELECT date_diff('day',
  DATE '2024-01-01', DATE '2024-06-15');

-- Truncation & bucketing
SELECT date_trunc('month',
  TIMESTAMPT '2024-06-15 10:30:00');
SELECT time_bucket(INTERVAL '1 hour',
  TIMESTAMPT '2024-06-15 10:35:00');

-- Formatting & parsing
SELECT strftime(NOW(), '%Y-%m-%d %H:%M');
SELECT strptime('2024-01-15', '%Y-%m-%d')::DATE;
```

Aggregate Functions

```
SELECT count(*), count(DISTINCT col),
  sum(val), avg(val) FROM t;
SELECT min(val), max(val), median(val) FROM t;

-- String & list aggregation
SELECT string_agg(name, ' '
  ORDER BY name) FROM t;
SELECT list(name ORDER BY name) FROM t;

-- Statistical
-- 95th percentile
SELECT quantile_cont(val, 0.95) FROM t;
SELECT approx_count_distinct(user_id) FROM t;
-- v1.4+
SELECT approx_top_k(product, 10) FROM orders;

-- argmin / argmax
-- Name of youngest / highest paid
SELECT arg_min(name, age) FROM users;
SELECT arg_max(name, salary) FROM users;

-- Histogram
SELECT histogram(age) FROM users;
```

Window Functions & QUALIFY

```
-- Running calculations
SELECT *, sum(val) OVER (ORDER BY date
  ROWS UNBOUNDED PRECEDING) AS running_total
FROM t;

-- QUALIFY: filter on window results
SELECT * FROM t
  QUALIFY row_number() OVER (
    PARTITION BY group_col
    ORDER BY score DESC) = 1;

-- FILL: gap-filling window function (v1.4+)
SELECT ts, FILL(value) OVER (ORDER BY ts)
FROM sensor_data;
```

PIVOT / UNPIVOT

```
PIVOT sales ON product USING sum(amount);
PIVOT sales ON product IN ('A', 'B', 'C')
  USING sum(amount) GROUP BY year;
UNPIVOT monthly_data
  ON jan, feb, mar
  INTO NAME month VALUE amount;
```

Advanced Joins

```
-- SEMI JOIN (rows from left that have a match)
SELECT * FROM a SEMI JOIN b ON a.id = b.id;

-- ANTI JOIN (rows from left with NO match)
SELECT * FROM a ANTI JOIN b ON a.id = b.id;

-- LATERAL JOIN (subquery refs preceding tables)
SELECT * FROM a, LATERAL (
  SELECT * FROM b
  WHERE b.a_id = a.id LIMIT 3);

-- ASOF JOIN (match nearest value)
SELECT * FROM trades ASOF JOIN quotes
  ON trades.ticker = quotes.ticker
  AND trades.ts >= quotes.ts;

-- POSITIONAL JOIN (by row position)
SELECT * FROM a POSITIONAL JOIN b;
```

CTEs (Common Table Expressions)

```
WITH cte AS (SELECT * FROM t WHERE val > 0)
SELECT * FROM cte;

-- Materialized CTE (force single evaluation)
WITH cte AS MATERIALIZED
  SELECT expensive_fn() AS val)
SELECT * FROM cte c1, cte c2;

-- Recursive CTE
WITH RECURSIVE counter AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM counter WHERE n < 100
)
SELECT * FROM counter;
```

ATTACH & Multi-Database

```
ATTACH 'other.duckdb' AS other_db;
ATTACH 'other.duckdb' AS other_db (READ_ONLY);

-- Cross-database queries
SELECT a.*, b.*
FROM db1.tbl a JOIN db2.tbl b ON a.id = b.id;

USE other_db;
DETACH other_db;

-- Attach MotherDuck
ATTACH 'md:my_database';
```

Extensions

```
-- Core extensions (often auto-loaded)
-- json, parquet, icu, tpch, tpcds
INSTALL https; LOAD https;

-- Community extensions (explicit install)
INSTALL h3 FROM community;

-- Update all / list installed
UPDATE EXTENSIONS;
FROM duckdb_extensions() WHERE installed;

-- Core: https (HTTP/S3/GCS), json, parquet,
-- icu, excel, spatial, delta, iceberg
-- Community: h3, sqllite_scanner, postgres_scanner
```

System & Introspection

```
-- Show table schema
DESCRIBE my_table;
-- Quick stats (min, max, nulls, distinct)
SUMMARIZE my_table;

-- List all tables in current database
SELECT * FROM duckdb_tables();
-- Column metadata (name, type, nullable)
SELECT * FROM duckdb_columns()
  WHERE table_name = 'my_table';

-- Query plan with execution stats
EXPLAIN ANALYZE
  SELECT * FROM t WHERE id > 100;

-- Configuration
SET memory_limit = '4GB';
SET threads = 4;
SELECT current_database(), current_schema();
-- Storage size on disk
PRAGMA database_size;
```

PART 2: WHAT'S NEW (V1.0 → V1.5)

v1.0 — June 2024 (Stability Release)

First stable release. Backward-compatible storage format and SQL dialect going forward.

v1.1 — September 2024

```
-- Named parameters
SELECT my_func(param => 'value');
-- query() / query_table()
SELECT * FROM query_table('my_table');
SELECT approx_top_k(product_name, 10)
FROM orders;
-- top-N min_by/max_by
SELECT min_by(name, age, 3) FROM users;
SELECT html_escape('<div>');
SELECT url_encode('hello world');
EXPLAIN (FORMAT JSON) SELECT * FROM t;
COPY t TO 'out/' (FORMAT PARQUET,
  PARTITION_BY (year), RETURN_FILES);
```

Also: Brotli compression, GeoParquet, JSON in Parquet, sha1(), configurable CTE materialization.

v1.2 — November 2024

```
SELECT * RENAME (name AS user_name)
FROM users;
ALTER TABLE t ADD PRIMARY KEY (id);
ALTER TABLE t ALTER col TYPE INTEGER
USING col::INTEGER;
SELECT * FROM t
WHERE (a, b) IN (SELECT x, y FROM other);
PIVOT sales ON product
  USING sum(amount), count(*);
CREATE TABLE t (a, b, c) AS SELECT 1, 2, 3;
```

Also: ZSTD/RoaringBitmap compression, Bloom filters in Parquet, max_temp_directory_size, filesystem access controls, CLI syntax highlighting.

v1.3 — January 2025

```
-- Time travel
SELECT * FROM my_table
  AT (TIMESTAMP => '2025-01-01');
ATTACH OR REPLACE 'new.duckdb' AS my_db;
ALTER TABLE t
  SET PARTITIONED BY (region, year);
ALTER TABLE t
  SET SORTED BY (created_at DESC);
ALTER TABLE t ALTER col
  ADD FIELD new_field INT;
CREATE OR REPLACE TYPE mood
  AS ENUM ('sad', 'happy', 'neutral');
-- TRI: NULL instead of error
SELECT TRY(1 / 0);
-- UUID v7 (time-sortable)
SELECT uuid();
```

Also: list_reduce() with initial value, json_each()/json_tree(), AVG for intervals, Float16 Parquet.

v1.4 — February 2025

```
-- MERGE INTO (full SQL standard)
MERGE INTO target USING source
  ON target.id = source.id
WHEN MATCHED THEN
  UPDATE SET val = source.val
WHEN NOT MATCHED THEN
  INSERT VALUES (source.id, source.val)
WHEN MATCHED AND source.deleted THEN DELETE;

SELECT val, ordinality
  FROM unnest(['a', 'b', 'c']) WITH ORDINALITY;

-- VARIANT type
CREATE TABLE events (data VARIANT);
SELECT variant_typeof(data),
  variant_extract(data, 'INTEGER')
FROM events;
```

```
-- BIGNUM: arbitrary precision
SELECT 9999999999999999::BIGNUM + 1;
-- FILL: gap-filling window function
SELECT ts, FILL(value) OVER (ORDER BY ts)
FROM sensor_data;
```

Also: COPY FORMAT BLOB, expression in COPY file targets, CTR/CBC encryption, make_timestamp_ms().

v1.5 — March 2026

```
-- GEOMETRY as built-in type
CREATE TABLE places (
  name TEXT, geom GEOMETRY);
CREATE TABLE geo_data (
  geom GEOMETRY('OGC:CRS84'));

-- read duckdb(): read/glob DuckDB databases
SELECT * FROM read_duckdb('backup_*.duckdb',
  table_name := 'users');
```

```
-- Lambda syntax (arrow deprecated)
SELECT list_transform([1,2,3],
  lambda x: x * 2);
```

```
SELECT array_intersect([1,2,3], [2,3,4]);
SELECT parse_formatted_bytes('1.5 GB');
ALTER DATABASE my_db RENAME TO new_name;
```

Also: VARIANT shredded Parquet, non-blocking checkpoints, ODBC scanner, Azure write, common subplan elimination, ~30-60% smaller extensions.

PART 3: CLI TIPS & TRICKS

CLI: Output Modes

```
.mode duckbox -- compact table (default)
.mode line   -- one col per line (wide/JSON)
.mode csv    -- CSV output
.mode json   -- JSON output
.mode markdown -- markdown table
```

CLI: Dot Commands

```
.tables          -- list all tables
.schema my_table -- show schema
.open my_db.duckdb -- open database
.read script.sql  -- run SQL file
.last            -- re-render prev result
SELECT * FROM _;  -- prev result set
.output results.csv -- export to file
.timer on         -- timer on/off
```

CLI: bar() — Inline Charts

```
-- Visual bar chart in terminal
SELECT name, score,
  bar(score, 0, 100, 20) AS chart
FROM students ORDER BY score DESC;
-- ██████████░░░░░░
```

CLI: Settings & Colors

```
SET enable_progress_bar = true;
SET enable_highlight = true;
-- Max rows / column width
SET max_rows = 40;
SET max_width = 80;

-- Custom colors (v1.5+)
SET highlight_keyword_color = 'blue';
SET highlight_string_color = 'green';
SET highlight_comment_color = 'gray';
SET highlight_error_color = 'red';
-- Supported: red, green, blue, yellow,
-- cyan, magenta, white, gray, bold, underline
```